Numerical Methods: Homework 4

B13902022 Yu-Chi, Lai

April 28, 2026

1 Introduction

This report details the implementation and optimization of Sparse-Dense Matrix Multiplication ($C = AB$). The sparse matrix A was implemented utilizing both Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) formats. The dense matrices B and C were stored in a 1D row-major layout. The custom algorithms (sequential and OpenMP parallelized) were benchmarked against the Intel oneMKL library across a strict set of matrix shapes. For all tests, the total number of non-zero elements (NNZ) in A was strictly maintained at $32 \times m$.

2 Experimental Setup

The full source code is in the appendix (the last few pages).

The benchmarks were compiled and executed locally with the following system configuration:

- **OS:** Arch Linux x86_64 (Kernel: Linux 6.19.14)
- **CPU:** Intel Core Ultra 5 125H (18 Cores) @ 4.50 GHz
- **RAM:** 16 GiB
- **Compiler:** GCC with OpenMP (`-fopenmp`)
- **Library:** Intel oneMKL (GNU Threading Layer, `lp64`)

The source code was compiled using the following command to link the threading and math libraries properly:

```
source /opt/intel/oneapi/setvars.sh
gcc -fopenmp -m64 -I"${MKLROOT}/include" new_spmm.c \
    -L"${MKLROOT}/lib/intel64" \
    -lmkl_intel_lp64 -lmkl_gnu_thread -lmkl_core \
    -lpthread -lm -ldl -o new_spmm
```

The testing is done by the following command (with different arguments):

```
./new_spmm <m_rows_A> <k_cols_A_rows_B> <n_cols_B>
```

where m is the number of rows of A, k is the number of cols A (number of rows of B), k is the number of rows of B.

3 Complexity Analysis & OpenMP Strategy

3.1 Algorithmic Complexity

Standard dense matrix multiplication operates with a time complexity of $O(m \cdot k \cdot n)$. By exploiting the sparsity of A, we eliminate all operations involving zero values. For each non-zero element in A, exactly n multiplications and additions are performed (iterating across the columns of B). Given the strict assignment constraint of $NNZ = 32m$, the time complexity becomes $O(32 \cdot m \cdot n)$, which simplifies to a linear bound of $O(m \cdot n)$ with respect to the dense matrix dimensions.

3.2 Parallelization Strategy

Selected Format: Compressed Sparse Row (CSR)
The CSR format was chosen for OpenMP parallelization (`#pragma omp parallel for`) to avoid race conditions and expensive atomic locks. In the CSR implementation, the outer loop iterates over the rows of A. Since each row of A independently computes a distinct row of C, we can safely distribute the workload across multiple hardware threads without write conflicts.

Attempting to parallelize the CSC format over its outer loop distributes the columns of A. Multiple threads processing different columns would inevitably attempt to accumulate their partial products into the exact same memory locations in C simultaneously, requiring atomic operations that severely degrade parallel performance.

4 Benchmark Results

The algorithms were benchmarked on A configurations of (n, n), $(4n, n)$, and $(n, 4n)$, with B column sizes of 64 and 512. The results in Table 1 are measured in seconds.

Table 1: SpMM Execution Time (Seconds) across Benchmark Shapes

Matrix Configurations		**CSR Format**			**CSC Format**	
Shape of $A(m \times k)$	**Cols of $B(n)$**	**Custom (Seq)**	**Custom (OMP)**	**oneMKL**	**Custom (Seq)**	**oneMKL**
(2048×2048)	64	0.0479	0.0098	0.0002	0.0599	0.0359
(2048×2048)	512	0.3864	0.0663	0.0056	0.3817	0.0773
(32768×32768)	64	0.8037	0.1351	0.0158	0.7820	0.6496
(32768×32768)	512	5.8841	0.8434	0.0319	4.4721	3.6203
(8192×2048)	64	0.1903	0.0313	0.0008	0.1868	0.1745
(8192×2048)	512	1.5271	0.2051	0.0245	1.4916	0.8153
(131072×32768)	64	3.1581	0.4691	0.0651	3.1940	1.7683
(131072×32768)	512	18.130	3.1545	0.1437	23.442	17.188
(2048×8192)	64	0.0473	0.0101	0.0004	0.0499	0.0318
(2048×8192)	512	0.3856	0.0655	0.0040	0.3761	0.0562
(32768×131072)	64	0.7873	0.1579	0.0093	0.7501	0.7872
(32768×131072)	512	5.3717	0.8001	0.0958	5.8568	1.0142

5 Performance Comparison and Discussion

The benchmark data yields several distinct architectural observations regarding memory layouts, multithreading, and library-level optimizations.

1. OpenMP Multithreading Scaling: The custom OpenMP CSR implementation demonstrated excellent scaling across all workloads. On the heaviest workload (131072×32768, $n = 512$), OpenMP completed the execution in 3.15 seconds compared to the sequential 18.13

seconds. This translates to an approximate 5.7× speedup, confirming that distributing row-wise matrix multiplications effectively utilizes the Intel Core Ultra 5's multicore architecture.

2. Intel oneMKL Hardware Dominance (CSR): As expected, Intel oneMKL's CSR implementation vastly outperformed the custom implementations. On the 32768×32768 ($n = 512$) benchmark, oneMKL completed in just $0.0319s$ compared to the custom OpenMP's $0.8434s$. This delta highlights oneMKL's reliance on highly optimized Inspector-Executor paradigms, cache blocking, and AVX SIMD vectorization, which maximize memory bandwidth and compute cycles far beyond standard C-loops.

3. The oneMKL CSC Anomaly: The data exposes a massive inefficiency within oneMKL when dealing with CSC matrices multiplying against row-major dense matrices. For instance, in the $(131072 \times 32768, n = 512)$ case, oneMKL CSR took a mere $0.143s$, whereas oneMKL CSC took a staggering $17.188s$—making it over 100× slower than its CSR counterpart, and significantly slower than our custom OpenMP CSR routine ($3.15s$). This suggests that `mkl_sparse_s_mm` is highly unoptimized for this specific format-layout pairing, likely forcing the library to perform an expensive internal matrix transposition prior to the multiplication step.

A Source Code (new_spmm.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <mkl.h>

#define ABS(x) ((x) < 0.0f ? -(x) : (x))
#define eps 1e-2

// Unified structure for both CSR and CSC formats
typedef struct { int r, c, nnz, is_csr; float *v; int *idx, *ptr; } SpMat;

// Unified SpMM function for both CSR and CSC
void spmm(SpMat *A, float *B, float *C, int n, int use_omp) {
    for (int i = 0; i < A->r * n; i++) C[i] = 0.0f;

    if (A->is_csr) {
        #pragma omp parallel for if(use_omp)
        for (int i = 0; i < A->r; i++) {
            for (int p = A->ptr[i]; p < A->ptr[i+1]; p++) {
                for (int j = 0; j < n; j++)
                    C[i*n + j] += A->v[p] * B[A->idx[p]*n + j];
            }
        }
    } else {
        for (int k = 0; k < A->c; k++) {
            for (int p = A->ptr[k]; p < A->ptr[k+1]; p++) {
                for (int j = 0; j < n; j++)
                    C[A->idx[p]*n + j] += A->v[p] * B[k*n + j];
            }
        }
    }
}

// Unified random matrix generator (NNZ = 32 * m constraint)
void gen_sp(SpMat *A, int m, int k, int is_csr) {
    int nnz = 32 * m, out_dim = is_csr ? m : k;
    A->r = m; A->c = k; A->nnz = nnz; A->is_csr = is_csr;
    A->v = malloc(nnz * sizeof(float)); A->idx = malloc(nnz * sizeof(int));
    A->ptr = calloc((out_dim + 1), sizeof(int));

    int cur = 0, rem = nnz % out_dim, base = nnz / out_dim;
    for (int i = 0; i < out_dim; i++) {
        int n_elem = base + (i < rem ? 1 : 0);
        for (int p = 0; p < n_elem; p++) {
            A->v[cur] = (rand() % 100) / 10.0f;
            A->idx[cur++] = p * (is_csr ? k : m) / n_elem;
        }
        A->ptr[i + 1] = cur;
    }
```

```
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        printf("Usage: %s <m_rows_A> <k_cols_A_rows_B> <n_cols_B>\n",
        ↪  argv[0]);
        return 1;
    }

    int m = atoi(argv[1]), k_dim = atoi(argv[2]), k_cols = atoi(argv[3]);

    // Memory allocation
    float *B = malloc(k_dim * k_cols * sizeof(float));
    float *C_csr_seq = malloc(m * k_cols * sizeof(float));
    float *C_csr_omp = malloc(m * k_cols * sizeof(float));
    float *C_csc_seq = malloc(m * k_cols * sizeof(float));
    float *C_mkl_csr = malloc(m * k_cols * sizeof(float));
    float *C_mkl_csc = malloc(m * k_cols * sizeof(float));

    for (int i = 0; i < k_dim * k_cols; i++) B[i] = (float)(rand() % 10) /
    ↪  10.0f;

    SpMat A_csr, A_csc;
    gen_sp(&A_csr, m, k_dim, 1);
    gen_sp(&A_csc, m, k_dim, 0);

    // Custom Benchmarks
    double start = omp_get_wtime(); spmm(&A_csr, B, C_csr_seq, k_cols, 0);
    printf("CSR (Seq): %f s\n", omp_get_wtime() - start);

    start = omp_get_wtime(); spmm(&A_csr, B, C_csr_omp, k_cols, 1);
    printf("CSR (OMP): %f s\n", omp_get_wtime() - start);

    start = omp_get_wtime(); spmm(&A_csc, B, C_csc_seq, k_cols, 0);
    printf("CSC (Seq): %f s\n", omp_get_wtime() - start);

    // Intel oneMKL Benchmarks
    sparse_matrix_t mkl_csr, mkl_csc;
    struct matrix_descr descr = {SPARSE_MATRIX_TYPE_GENERAL, 0, 0};

    mkl_sparse_s_create_csr(&mkl_csr, SPARSE_INDEX_BASE_ZERO, m, k_dim,
                            A_csr.ptr, A_csr.ptr+1, A_csr.idx, A_csr.v);
    start = omp_get_wtime();
    mkl_sparse_s_mm(SPARSE_OPERATION_NON_TRANSPOSE, 1.0f, mkl_csr, descr,
                    SPARSE_LAYOUT_ROW_MAJOR, B, k_cols, k_cols, 0.0f,
                    ↪  C_mkl_csr, k_cols);
    printf("oneMKL CSR: %f s\n", omp_get_wtime() - start);

    mkl_sparse_s_create_csc(&mkl_csc, SPARSE_INDEX_BASE_ZERO, m, k_dim,
                            A_csc.ptr, A_csc.ptr+1, A_csc.idx, A_csc.v);
    start = omp_get_wtime();
    mkl_sparse_s_mm(SPARSE_OPERATION_NON_TRANSPOSE, 1.0f, mkl_csc, descr,
```

```
                    SPARSE_LAYOUT_ROW_MAJOR, B, k_cols, k_cols, 0.0f,
                    ↪ C_mkl_csc, k_cols);
    printf("oneMKL CSC: %f s\n", omp_get_wtime() - start);

    // Correctness Verification
    int seq_pass = 1, omp_pass = 1, csc_pass = 1;
    for (int i = 0; i < m * k_cols; i++) {
        if (ABS(C_csr_seq[i] - C_mkl_csr[i]) > eps) seq_pass = 0;
        if (ABS(C_csr_omp[i] - C_mkl_csr[i]) > eps) omp_pass = 0;
        if (ABS(C_csc_seq[i] - C_mkl_csc[i]) > eps) csc_pass = 0;
    }

    printf("\n--- Verification ---\n");
    printf("CSR Sequential: %s\n", seq_pass ? "PASSED" : "FAILED");
    printf("CSR OpenMP:     %s\n", omp_pass ? "PASSED" : "FAILED");
    printf("CSC Sequential: %s\n", csc_pass ? "PASSED" : "FAILED");

    // Cleanup
    mkl_sparse_destroy(mkl_csr);
    mkl_sparse_destroy(mkl_csc);
    free(A_csr.v); free(A_csr.idx); free(A_csr.ptr);
    free(A_csc.v); free(A_csc.idx); free(A_csc.ptr);
    free(B); free(C_csr_seq); free(C_csr_omp); free(C_csc_seq);
    free(C_mkl_csr); free(C_mkl_csc);

    return 0;
}
```